

Kotlin第一章第二节教学讲义（类与流程控制进阶）

各位同学，上节课我们入门了Kotlin的基本语法和简单控制流程，这节课咱们重点攻克“类”这个核心概念——它是面向对象编程的基石，就像“汽车图纸”一样，能帮我们批量创建有相同特性的“实体”。同时我们会补充流程控制的进阶用法，让代码逻辑更灵活。全程代码+通俗解释，零基础也能轻松掌握！

一、流程控制语句进阶：让逻辑更精准

上节课学了if、when、for、while的基本用法，这节课补充“循环跳出”和“循环嵌套”，解决实际开发中“需要提前结束循环”“多层循环控制”的问题。

1. 循环跳出：break与continue

就像看书时“跳过某一页（continue）”和“直接合上书（break）”，这两个关键字能精准控制循环执行：

- **break**：立即终止当前循环，跳出循环体
- **continue**：跳过当前循环的剩余代码，直接开始下一次循环

代码示例：筛选1-10中的偶数

Code block

```
1  fun main() {
2      // 示例1: 用break找到第一个大于5的数
3      for (i in 1..10) {
4          if (i > 5) {
5              println("第一个大于5的数是: $i")
6              break // 找到后直接终止循环, 后续i=6-10不再执行
7          }
8      }
9      // 输出: 第一个大于5的数是: 6
10
11     // 示例2: 用continue跳过奇数, 只打印偶数
12     for (i in 1..10) {
13         if (i % 2 != 0) {
14             continue // 是奇数就跳过, 不执行下面的println
15         }
16         println("偶数: $i")
17     }
18 }
```

```
17     }
18     // 输出: 偶数: 2  偶数: 4  偶数: 6  偶数: 8  偶数: 10
19 }
```

2. 循环嵌套与标签跳出

当循环里再套循环（比如“遍历表格的行和列”），可以给外层循环加**标签**，实现“精准跳出外层循环”。标签格式是“标签名@”。

代码示例：找到“行=2，列=3”时终止外层循环

Code block

```
1 fun main() {
2     // 给外层循环加标签: outer@
3     outer@ for (row in 1..3) { // 行: 1-3
4         for (col in 1..4) { // 列: 1-4
5             println("当前位置: 行$row, 列$col")
6             if (row == 2 && col == 3) {
7                 break@outer // 直接跳出外层循环, 而不是只跳出内层
8             }
9         }
10    }
11    // 执行结果: 行1列1-4 → 行2列1-3, 之后循环终止
12 }
```

二、类：Kotlin的“实体图纸”

生活中“汽车”有统一特性（轮子数、颜色）和功能（跑、刹车），Kotlin中“类”就是描述这些“特性和功能”的图纸，用 `class` 关键字定义。根据图纸造出来的“具体汽车”，叫做“对象”（实例）。

1. 类的基本定义与使用

语法: `class 类名(属性1: 类型, 属性2: 类型) { // 类体: 放功能(方法) }`

代码示例：定义“汽车类”并创建对象

Code block

```
1 // 1. 定义类: 汽车图纸
2 class Car(
3     val brand: String, // 品牌 (不可变, val)
4     var color: String, // 颜色 (可变, var)
5     val wheelCount: Int = 4 // 轮子数 (默认值4, 创建对象时可省略)
6 ) {
```

```
7 // 类的功能（方法）：跑
8 fun run() {
9     println("$brand $color款汽车，正在路上跑！")
10 }
11
12 // 类的功能（方法）：改颜色（简化写法，一行代码可省略大括号）
13 fun changeColor(newColor: String) = color = newColor
14 }
15
16 // 2. 使用类：根据图纸造对象（实例化）
17 fun main() {
18     // 创建对象1：传全参数
19     val myCar = Car("特斯拉", "红色")
20     // 调用对象的属性和方法
21     println("我的车：${myCar.brand}, 颜色：${myCar.color}")
22     myCar.run() // 调用方法：输出“特斯拉 红色款汽车，正在路上跑！”
23
24     // 创建对象2：省略默认参数（轮子数）
25     val yourCar = Car("比亚迪", "蓝色")
26     yourCar.changeColor("黑色") // 改颜色
27     println("你的车新颜色：${yourCar.color}") // 输出：黑色
28 }
```

2. 属性：类的“特性”（变量）

类中的属性就是“实体的特性”，比如汽车的品牌、颜色。Kotlin的属性比Java简洁，支持“默认值”“自动生成getter/setter”，不用手动写繁琐代码。

(1) 属性的两种类型

关键字	特点	生成的方法	使用场景
val（只读）	赋值后不可改	只生成getter（获取值）	固定特性（品牌、
var（可写）	可修改值	生成getter+setter（设置值）	可变特性（颜色、

(2) 自定义getter/setter（进阶）

当需要对属性的“获取/设置”加逻辑（比如“年龄不能为负数”），可以自定义getter/setter：

Code block

```
1 class Person(name: String) {
2     // 普通属性
3     val name: String = name
```

```

4
5    // 自定义setter的属性：年龄
6    var age: Int = 0
7        set(value) {
8            // value是要设置的新值
9            if (value < 0) {
10                println("年龄不能为负数，默认设为0")
11                field = 0 // field是“幕后字段”，代表属性本身的值
12            } else {
13                field = value
14            }
15        }
16
17    // 自定义getter的属性：是否成年（根据年龄计算）
18    val isAdult: Boolean
19        get() = age >= 18
20 }
21
22 fun main() {
23     val person = Person("小明")
24     person.age = -5 // 输出：年龄不能为负数，默认设为0
25     println("年龄：${person.age}，是否成年：${person.isAdult}") // 0, false
26
27     person.age = 20
28     println("年龄：${person.age}，是否成年：${person.isAdult}") // 20, true
29 }

```

三、构造器：创建对象的“入口”

构造器是“创建对象时初始化属性”的特殊方法，Kotlin分两种：**主构造器**（类名后直接写，简洁）和**次构造器**（类体中用 `constructor` 定义，解决“多参数组合”场景）。

1. 主构造器（常用）

直接跟在类名后，适合简单初始化，比如之前的Car类：`class Car(val brand: String, var color: String)`

如果需要在主构造器中加初始化逻辑（比如验证参数），用 `init` 代码块：

Code block

```

1    class Phone(val brand: String, val price: Double) {
2        // init代码块：主构造器的初始化逻辑，创建对象时执行
3        init {
4            require(price > 0) { "价格必须大于0" } // 验证参数，不满足则崩溃并提示
5            println("创建了$brand手机，价格：$price")

```

```

6      }
7  }
8
9  fun main() {
10     val phone1 = Phone("苹果", 5999.0) // 正常执行, 输出提示
11     // val phone2 = Phone("小米", -100.0) // 崩溃, 提示“价格必须大于0”
12 }

```

2. 次构造器（备用）

当创建对象有多种“参数组合”时用次构造器，必须用 `this` 关联主构造器（或其他次构造器）：

Code block

```

1  class Student(val name: String) {
2      var age: Int = 0
3      var className: String = "未分班"
4
5      // 次构造器1: name + age
6      constructor(name: String, age: Int) : this(name) {
7          this.age = age
8      }
9
10     // 次构造器2: name + age + className
11     constructor(name: String, age: Int, className: String) : this(name, age) {
12         this.className = className
13     }
14
15     fun introduce() {
16         println("我是$name, $age岁, 在$className")
17     }
18 }
19
20 fun main() {
21     val s1 = Student("小红") // 主构造器
22     val s2 = Student("小李", 15) // 次构造器1
23     val s3 = Student("小张", 16, "高一(1)班") // 次构造器2
24
25     s3.introduce() // 输出: 我是小张, 16岁, 在高一(1)班
26 }

```

四、继承与覆盖：站在“已有类”的肩膀上

生活中“电动车”是“汽车”的一种，它继承了汽车的“轮子、跑”特性，又多了“用电”的特殊功能。Kotlin中继承用 `:` 实现，但有两个规则：

- 父类（被继承的类）必须用 `open` 修饰（默认不能被继承，保证安全性）
- 子类（继承的类）覆盖父类的方法/属性时，必须用 `override` 修饰

1. 类的继承

Code block

```
1  // 父类：汽车（必须加open才能被继承）
2  open class Car(val brand: String) {
3      // 父类方法（加open才能被覆盖）
4      open fun run() {
5          println("$brand汽车用发动机跑")
6      }
7  }
8
9  // 子类：电动车（继承Car）
10 class ElectricCar(brand: String, val batteryCapacity: Int) : Car(brand) {
11     // 覆盖父类的run方法（必须加override）
12     override fun run() {
13         super.run() // 调用父类的run方法
14         println("这是电动车，电池容量$batteryCapacity，用电跑")
15     }
16
17     // 子类特有的方法
18     fun charge() {
19         println("电动车正在充电")
20     }
21 }
22
23 fun main() {
24     val tesla = ElectricCar("特斯拉", 100)
25     tesla.run() // 先执行父类run，再执行子类run
26     tesla.charge() // 调用子类特有方法
27
28     // 输出：
29     // 特斯拉汽车用发动机跑
30     // 这是电动车，电池容量100，用电跑
31     // 电动车正在充电
32 }
```

2. 覆盖属性

子类也能覆盖父类的属性，比如“电动车的轮子数默认是4，但可以改成特殊值”：

Code block

```
1  open class Car {
2      open val wheelCount: Int = 4 // 父类属性，加open
3  }
4
5  class SpecialCar : Car() {
6      // 覆盖父类属性
7      override val wheelCount: Int = 6
8  }
9
10 fun main() {
11     val car = Car()
12     val specialCar = SpecialCar()
13     println("普通车车轮: ${car.wheelCount}") // 4
14     println("特殊车车轮: ${specialCar.wheelCount}") // 6
15 }
```

五、类的修饰符：控制类的“访问权限”

修饰符就像“房间的门牌号”，控制其他类能否访问这个类、属性或方法。Kotlin有4种常用修饰符，从开放到严格排序：

修饰符	访问范围	使用场景
public（默认）	整个项目都能访问	公开的工具类、组件
internal	当前模块（比如APP模块）能访问	模块内共用的类，不对外暴露
protected	当前类和子类能访问	父类给子类的“专属特性”
private	只有当前类内部能访问	类的“私有逻辑”，比如内部计算的临时变量

代码示例：修饰符的使用

Code block

```
1  // internal修饰类：只能在当前模块访问
2  internal class Tool {
3      // public方法（默认）：模块内都能调用
4      fun publicMethod() = println("公开方法")
5
6      // protected方法：只有Tool类和它的子类能调用
7      protected fun protectedMethod() = println("保护方法")
8  }
```

```

9      // private方法：只有Tool类内部能调用
10     private fun privateMethod() = println("私有方法")
11
12     fun usePrivate() {
13         privateMethod() // 类内部可以调用私有方法
14     }
15 }
16
17 // 子类继承Tool
18 class MyTool : Tool() {
19     fun useProtected() {
20         protectedMethod() // 子类可以调用保护方法
21     }
22 }
23
24 fun main() {
25     val tool = Tool()
26     tool.publicMethod() // 正常调用
27     tool.usePrivate() // 正常调用（间接调用私有方法）
28     // tool.protectedMethod() // 报错：不能访问保护方法
29     // tool.privateMethod() // 报错：不能访问私有方法
30
31     val myTool = MyTool()
32     myTool.useProtected() // 子类正常调用
33 }

```

六、内部类与嵌套类：类的“内部房间”

当一个类只给另一个类“专属服务”时，就可以把它定义在另一个类内部，分成两种：

1. 嵌套类（静态内部类）

用 `class` 直接定义在类内部，**不依赖外部类对象**，就像“汽车说明书”（嵌套类）和“汽车”（外部类）——说明书不依赖具体汽车存在。

Code block

```

1  // 外部类：汽车
2  class Car(val brand: String) {
3      // 嵌套类：汽车说明书（不依赖外部类对象）
4      class Manual {
5          fun showInfo() {
6              println("这是汽车的使用说明书")
7              // 不能直接访问外部类的brand属性
8          }
9      }

```



```

10  }
11
12  fun main() {
13      // 直接通过外部类访问嵌套类，不用创建外部类对象
14      val manual = Car.Manual()
15      manual.showInfo() // 输出：这是汽车的使用说明书
16  }

```

2. 内部类（非静态内部类）

用 `inner` 关键字定义，**依赖外部类对象**，就像“汽车发动机”（内部类）和“汽车”（外部类）——发动机必须装在具体汽车上才能用，能访问外部类的属性。

Code block

```

1  // 外部类：汽车
2  class Car(val brand: String) {
3      // 内部类：发动机（加inner，依赖外部类对象）
4      inner class Engine {
5          fun start() {
6              // 可以直接访问外部类的brand属性
7              println("$brand汽车的发动机启动了")
8          }
9      }
10 }
11
12 fun main() {
13     // 必须先创建外部类对象，才能创建内部类对象
14     val car = Car("比亚迪")
15     val engine = car.Engine() // 通过外部类对象获取内部类
16     engine.start() // 输出：比亚迪汽车的发动机启动了
17 }

```

总结与课后作业

1. 核心知识点回顾

- **流程控制**：break（终止循环）、continue（跳过本次）、标签（控制嵌套循环）
- **类与对象**：class定义类，val/var定义属性，fun定义方法，对象是类的实例
- **构造器**：主构造器（类名后）+ init代码块，次构造器用constructor+this关联
- **继承**：父类open，子类:继承，override覆盖方法/属性，super调用父类逻辑
- **修饰符**：public（默认）>internal>protected>private，控制访问权限

- **内部类**：inner修饰（依赖外部类，能访问外部属性），嵌套类（不依赖）

2. 课后作业：实现“学生管理”功能

需求：

1. 定义父类 `Person`（open修饰），包含属性 `name`（val）、`age`（var，自定义setter防止负数），方法 `introduce()`（open修饰，打印“我是XX，XX岁”）
2. 定义子类 `Student` 继承 `Person`，新增属性 `studentId`（val）、`score`（var），覆盖 `introduce()` 方法，新增方法 `getGrade()`（用when判断分数等级：90+优秀，80-89良好，60-79及格，否则不及格）
3. 创建3个 `Student` 对象，用循环遍历并调用它们的 `introduce()` 和 `getGrade()` 方法

提示：用for循环遍历 `Student` 对象的集合，比如 `val students = listOf(student1, student2, student3)`

（注：文档部分内容可能由 AI 生成）